

Base de données Portfolio

Documentation complète · MCD · MLD · MPD · Schéma PostgreSQL

Auteure Salomé Souque	Stack Next.js · React · PostgreSQL · DBeaver	i18n Français / English	Version 1.0 — 2026
------------------------------	---	--------------------------------	---------------------------

Sommaire

1.

MCD — Modèle Conceptuel des Données

Entités, associations, cardinalités
2.

MLD — Modèle Logique des Données

Tables, clés, relations normalisées
3.

MPD — Modèle Physique des Données

SQL PostgreSQL exécutable
4.

Dictionnaire des données

Description colonne par colonne
5.

Points d'attention & bonnes pratiques

Questions clés pour l'implémentation

■ PK — Clé primaire (identifiant unique)	■ FK — Clé étrangère (référence)	■ UK — Valeur unique
---	---	-----------------------------

1. MCD — Modèle Conceptuel des Données

Le MCD représente les entités du monde réel et leurs relations, sans se préoccuper de l'implémentation technique. On identifie les **entités** (rectangles), leurs **attributs**, les **associations** (losanges/ovales) et les **cardinalités** (1,1 · 0,N · 1,N...).

Lecture des cardinalités : (1,1) = exactement un, (0,N) = zéro ou plusieurs, (1,N) = un ou plusieurs. La cardinalité se lit du côté de l'entité opposée.

Entité 1	Card.	Association	Card.	Entité 2	Note
PROJET	(1,1)	EST TRADUIT EN	(1,N)	TRADUCTION_PROJET	Un projet a au moins une traduction.
PROJET	(1,1)	CONTIENT	(0,N)	SECTION	Un projet peut avoir des sections.
PROJET	(1,1)	POSSEDE	(0,N)	IMAGE_PROJET	Un projet peut avoir des images.
PROJET	(0,N)	APPARTIENT A	(0,N)	CATEGORIE	Relation N:N — table de liaison.
PROJET	(0,N)	UTILISE	(0,N)	COMPETENCE	Relation N:N — table de liaison.
PROJET	(1,1)	A	(0,N)	LIEN	Un projet peut avoir des liens.
CATEGORIE	(1,1)	EST TRADUITE EN	(1,N)	TRADUCTION_CAT	Une catégorie a au moins une traduction.
COMPETENCE	(0,N)	APPARTIENT A	(1,1)	CAT_COMPETENCE	Une compétence appartient à une catégorie.
COMPETENCE	(1,1)	EST TRADUITE EN	(1,N)	TRADUCTION_SKILL	Une compétence a au moins une traduction.
COMPETENCE	(0,N)	APPARAÎT SUR	(0,N)	PAGE	Via page_key : 'home', 'skills'...
CAT_COMPETENCE	(1,1)	EST TRADUITE EN	(1,N)	TRAD_CAT_SKILL	La catégorie skill a ses traductions.
MEDIA	(1,1)	EST REFERENCE PAR	(0,N)	PAGE/COMPOSANT	Clé unique pour retrouver un media.
FORMATION	—	—	—	—	Entité autonome (timeline, statut).

Entités identifiées (16 au total)

Entité	Attributs principaux
PROJET	slug, cover_image, date, github_url, demo_url, featured, display_order
TRADUCTION_PROJET	locale, title, short_description, long_description
SECTION	locale, section_key, title, content, display_order
IMAGE_PROJET	file_path, alt_text, display_order
CATEGORIE	slug, color_hex, icon
TRADUCTION_CAT	locale, label
CAT_COMPETENCE	slug, color_hex

Entité	Attributs principaux
TRAD_CAT_COMPETENCE	locale, label
COMPETENCE	slug, icon_url, proficiency_level
TRADUCTION_SKILL	locale, label
PAGE_SKILL	page_key, display_order
FORMATION	degree, institution, start_year, end_year, is_current, display_order
MEDIA	key, file_path_light, file_path_dark, alt_text, media_type
LIEN	page_key, platform, url, locale

2. MLD — Modèle Logique des Données

Le MLD traduit le MCD en tables relationnelles. Les associations N:N deviennent des tables de liaison. Les clés étrangères (FK) matérialisent les relations. La contrainte ON DELETE CASCADE garantit l'intégrité référentielle : supprimer un projet supprime automatiquement ses traductions, sections, images, etc.

Table : projects

Colonne	Type	Contrainte
id	SERIAL	PK
slug	TEXT	UK
cover_image	TEXT	—
date	TEXT	—
github_url	TEXT	—
demo_url	TEXT	—
featured	BOOLEAN	—
display_order	INTEGER	—

Table : project_translations

Colonne	Type	Contrainte
id	SERIAL	PK
project_id	INTEGER	FK → projects(id)
locale	TEXT	—
title	TEXT	—
short_description	TEXT	—
long_description	TEXT	—

Table : project_sections

Colonne	Type	Contrainte
id	SERIAL	PK
project_id	INTEGER	FK → projects(id)
locale	TEXT	—
section_key	TEXT	—
title	TEXT	—
content	TEXT	—
display_order	INTEGER	—

Table : project_images

Colonne	Type	Contrainte
id	SERIAL	PK
project_id	INTEGER	FK → projects(id)
file_path	TEXT	—
alt_text	TEXT	—
display_order	INTEGER	—

Table : categories

Colonne	Type	Contrainte
id	SERIAL	PK
slug	TEXT	UK
color_hex	TEXT	—
icon	TEXT	—

Table : category_translations

Colonne	Type	Contrainte
id	SERIAL	PK
category_id	INTEGER	FK → categories(id)
locale	TEXT	—
label	TEXT	—

Table : project_categories

Colonne	Type	Contrainte
project_id	INTEGER	FK → projects(id) [PK composite]
category_id	INTEGER	FK → categories(id) [PK composite]

Table : skill_categories

Colonne	Type	Contrainte
id	SERIAL	PK
slug	TEXT	UK
color_hex	TEXT	—

Table : skill_category_translations

Colonne	Type	Contrainte
id	SERIAL	PK
skill_category_id	INTEGER	FK → skill_categories(id)
locale	TEXT	—
label	TEXT	—

Table : skills

Colonne	Type	Contrainte
id	SERIAL	PK
slug	TEXT	UK
skill_category_id	INTEGER	FK → skill_categories(id)
icon_url	TEXT	—
proficiency_level	INTEGER	0–100

Table : skill_translations

Colonne	Type	Contrainte
id	SERIAL	PK
skill_id	INTEGER	FK → skills(id)
locale	TEXT	—
label	TEXT	—

Table : page_skills

Colonne	Type	Contrainte
id	SERIAL	PK
page_key	TEXT	—
skill_id	INTEGER	FK → skills(id)
display_order	INTEGER	—

Table : project_skills

Colonne	Type	Contrainte
project_id	INTEGER	FK → projects(id) [PK composite]
skill_id	INTEGER	FK → skills(id) [PK composite]

Table : education

Colonne	Type	Contrainte
id	SERIAL	PK
degree	TEXT	—
institution	TEXT	—
start_year	INTEGER	—
end_year	INTEGER	NULL si en cours
is_current	BOOLEAN	—
display_order	INTEGER	—

Table : media

Colonne	Type	Contrainte
id	SERIAL	PK
key	TEXT	UK
file_path_light	TEXT	—
file_path_dark	TEXT	NULL si identique
alt_text	TEXT	—
media_type	TEXT	—

Table : links

Colonne	Type	Contrainte
id	SERIAL	PK
page_key	TEXT	NULL si lien projet
project_id	INTEGER	FK → projects(id) nullable
platform	TEXT	—
url	TEXT	—
locale	TEXT	NULL si universel


```
category_id INTEGER NOT NULL REFERENCES categories(id) ON DELETE CASCADE,  
locale      TEXT NOT NULL,  
label       TEXT NOT NULL,  
UNIQUE (category_id, locale)  
);
```

-- ■■■ PROJETS ■■■

```
CREATE TABLE projects (
  id          SERIAL PRIMARY KEY,
  slug        TEXT NOT NULL UNIQUE,
  cover_image TEXT,
  date        TEXT,
  github_url  TEXT,
  demo_url    TEXT,
  featured    BOOLEAN DEFAULT false,
  display_order INTEGER DEFAULT 0
);
```

```
CREATE TABLE project_translations (
    id SERIAL PRIMARY KEY,
    project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    locale TEXT NOT NULL,
    title TEXT NOT NULL,
    short_description TEXT,
    long_description TEXT,
    UNIQUE (project_id, locale)
);
```

```
CREATE TABLE project_sections (
  id SERIAL PRIMARY KEY,
  project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  locale TEXT NOT NULL,
  section_key TEXT NOT NULL,
  title TEXT,
  content TEXT,
  display_order INTEGER DEFAULT 0
);
```

```
CREATE TABLE project_images (
    id SERIAL PRIMARY KEY,
    project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    file_path TEXT NOT NULL,
    alt_text TEXT,
    display_order INTEGER DEFAULT 0
);
```

```
-- ■■ TABLES DE LIAISON N:N
```

```
CREATE TABLE project_categories (
    project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    category_id INTEGER NOT NULL REFERENCES categories(id) ON DELETE CASCADE,
    PRIMARY KEY (project_id, category_id)
);
```

```
CREATE TABLE project_skills (
```

```

    project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
    skill_id INTEGER NOT NULL REFERENCES skills(id) ON DELETE CASCADE,
    PRIMARY KEY (project_id, skill_id)
);
```

[illegible]

```
CREATE TABLE education (
    id SERIAL PRIMARY KEY,
    degree TEXT NOT NULL,
    institution TEXT NOT NULL,
    start_year INTEGER NOT NULL,
    end_year INTEGER,
    is_current BOOLEAN DEFAULT false,
    display_order INTEGER DEFAULT 0
);
```

```
CREATE TABLE media (
    id SERIAL PRIMARY KEY,
    key TEXT NOT NULL UNIQUE,
    file_path_light TEXT NOT NULL,
    file_path_dark TEXT,
    alt_text TEXT,
    media_type TEXT NOT NULL
);
```

```
CREATE TABLE links (
    id SERIAL PRIMARY KEY,
    page_key TEXT,
    project_id INTEGER REFERENCES projects(id) ON DELETE CASCADE,
    platform TEXT NOT NULL,
    url TEXT NOT NULL,
    locale TEXT
);
```

[illegible]

```
CREATE INDEX idx_project_translations_locale ON project_translations(project_id, locale);
CREATE INDEX idx_project_sections_locale ON project_sections(project_id, locale);
CREATE INDEX idx_skill_translations_locale ON skill_translations(skill_id, locale);
CREATE INDEX idx_category_translations_locale ON category_translations(category_id, locale);
CREATE INDEX idx_page_skills_page_key ON page_skills(page_key);
CREATE INDEX idx_projects_featured ON projects(featured);
CREATE INDEX idx_projects_display_order ON projects(display_order);
```

4. Dictionnaire des données

■ **PK** — Clé primaire (identifiant unique)

■ **FK** — Clé étrangère (référence)

■ **UK** — Valeur unique

projects

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
slug	TEXT	UK	Identifiant URL du projet. Utilisé dans /[locale]/projets/[slug].
cover_image	TEXT		Chemin relatif de l'image de couverture (ex: /images/projects/dashboard/cover.png).
date	TEXT		Date du projet au format YYYY-MM. Affiché sur la carte et le détail.
github_url	TEXT		URL du dépôt GitHub. NULL si projet privé.
demo_url	TEXT		URL de la démo en ligne. NULL si pas de démo.
featured	BOOLEAN		Si true, le projet apparaît dans la sélection de la page d'accueil.
display_order	INTEGER		Ordre d'affichage. Valeur basse = affiché en premier.

project_translations

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
project_id	INTEGER	FK	Référence vers projects(id). CASCADE à la suppression.
locale	TEXT		Code langue : 'fr' ou 'en'. Contrainte UNIQUE avec project_id.
title	TEXT		Titre traduit du projet.
short_description	TEXT		Description courte pour la carte projet (~120 caractères recommandés).
long_description	TEXT		Description longue pour la section 'Le Défi' de la page détail.

project_sections

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
project_id	INTEGER	FK	Référence vers projects(id).
locale	TEXT		Code langue : 'fr' ou 'en'.
section_key	TEXT		Clé fonctionnelle : 'challenge', 'solution', 'role', 'results', 'demo'...
title	TEXT		Titre de la section. Peut contenir du Markdown.
content	TEXT		Contenu de la section. Peut contenir du Markdown pour le rendu riche.
display_order	INTEGER		Ordre d'affichage des sections sur la page projet.

project_images

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
project_id	INTEGER	FK	Référence vers projects(id).
file_path	TEXT		Chemin relatif de l'image (ex: /images/projects/dashboard/aperçu-1.png).
alt_text	TEXT		Texte alternatif pour l'accessibilité (WCAG).
display_order	INTEGER		Ordre dans la galerie d'aperçus.

categories

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
slug	TEXT	UK	Identifiant neutre : 'ia', 'web', 'data-science', 'photos'...
color_hex	TEXT		Couleur hexadécimale des badges de filtre (ex: '#e96b99' pour Web).
icon	TEXT		Nom d'icône optionnel pour affichage dans les boutons de filtre.

skill_categories

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
slug	TEXT	UK	Identifiant neutre : 'ia', 'web', 'data-science', 'infra'...
color_hex	TEXT		Couleur commune à toutes les compétences de ce groupe. Ex: '#6c63ff' pour IA.

skills

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
slug	TEXT	UK	Identifiant neutre : 'python', 'react', 'power-bi', 'pytorch'...
skill_category_id	INTEGER	FK	Référence vers skill_categories(id). Détermine la couleur du tag.
icon_url	TEXT		URL ou chemin vers l'icône SVG de la compétence (optionnel).
proficiency_level	INTEGER		Niveau de 0 à 100. Alimente les barres de progression de la page compétences.

page_skills

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
page_key	TEXT		Page d'affichage : 'home' ou 'skills'. Extensible sans modifier le schéma.
skill_id	INTEGER	FK	Référence vers skills(id).
display_order	INTEGER		Ordre d'affichage du tag sur la page ciblée.

education

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
degree	TEXT		Intitulé du diplôme ou de la formation.
institution	TEXT		Nom de l'établissement (EM Lyon, Artefact...).
start_year	INTEGER		Année de début.
end_year	INTEGER		Année de fin. NULL si formation en cours (voir is_current).
is_current	BOOLEAN		Si true, affiche le badge 'En cours' dans la timeline.
display_order	INTEGER		Ordre dans la timeline (valeur basse = en haut).

media

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
key	TEXT	UK	Clé de référence unique : 'hero-illustration', 'portrait', 'logo'...
file_path_light	TEXT		Chemin vers la version thème clair.
file_path_dark	TEXT		Chemin vers la version thème sombre. NULL si le fichier est identique.
alt_text	TEXT		Texte alternatif pour l'accessibilité.
media_type	TEXT		Type de média : 'illustration', 'portrait', 'logo', 'icon'...

links

Colonne	Type	Clé	Description
id	SERIAL	PK	Identifiant auto-incrémenté.
page_key	TEXT		Page d'affichage : 'footer', 'home', 'about'... NULL si lien de projet.
project_id	INTEGER	FK	Référence vers projects(id). NULL si lien global (footer, nav).
platform	TEXT		Plateforme : 'linkedin', 'github', 'instagram', 'demo', 'cv', 'photo-portfolio'.
url	TEXT		URL complète du lien.
locale	TEXT		Code langue si le lien est spécifique à une langue. NULL sinon.

5. Points d'attention & bonnes pratiques

Ordre de création des tables	PostgreSQL refuse de créer une FK vers une table qui n'existe pas encore. Respecter l'ordre : skill_categories → skills → categories → projects → tables de liaison. Le MPD ci-dessus est déjà dans le bon ordre.
Ajouter une nouvelle langue	Il suffit d'insérer de nouvelles lignes dans toutes les tables _translations avec le nouveau code locale (ex: 'es' pour l'espagnol). Aucune modification de schéma nécessaire.
Ajouter un projet	1 ligne dans projects · 2 lignes dans project_translations (fr + en) · N lignes dans project_sections · N lignes dans project_images · lignes dans project_categories et project_skills.
Images : chemins relatifs vs absolus	Toujours stocker des chemins relatifs (/images/projects/...) et non des URLs absolues. Cela permet de changer de domaine ou d'hébergement sans toucher à la base de données.
Thème clair/sombre	La table media gère les deux variantes via file_path_light et file_path_dark. Dans Next.js, le composant lit le thème via useTheme() (next-themes) et sert le bon chemin.
Sécurité des requêtes SQL	Toujours utiliser des requêtes paramétrées (\$1, \$2...) avec la lib pg de Node.js. Ne jamais interpoler la locale ou le slug directement dans la chaîne SQL (risque d'injection).
Index de performance	Les index créés dans le MPD couvrent les jointures les plus fréquentes (locale, page_key, featured). Pour un portfolio (~20 projets), l'impact est minime mais c'est une bonne habitude.
Données de test (seed)	Créer un fichier seed.sql avec des données de test réalistes avant de commencer le front. Cela permet de développer l'interface sans avoir à saisir des données manuellement.
Variables d'environnement	Ne jamais coder en dur l'URL de connexion PostgreSQL. Utiliser process.env.DATABASE_URL dans un fichier .env (ajouté au .gitignore).
Sauvegarde de la base	Sur le serveur, planifier des exports réguliers avec pg_dump. La base de données n'est pas dans Git — elle doit être sauvegardée séparément.